

CHAPTER 2 | Data Models

What are Data Models?

Data Models are the abstract, conceptual framework that defines

- How data is represented
- Relationship between data
- Constraints on the data
- Operations that can be performed on the data

Types of Data Models

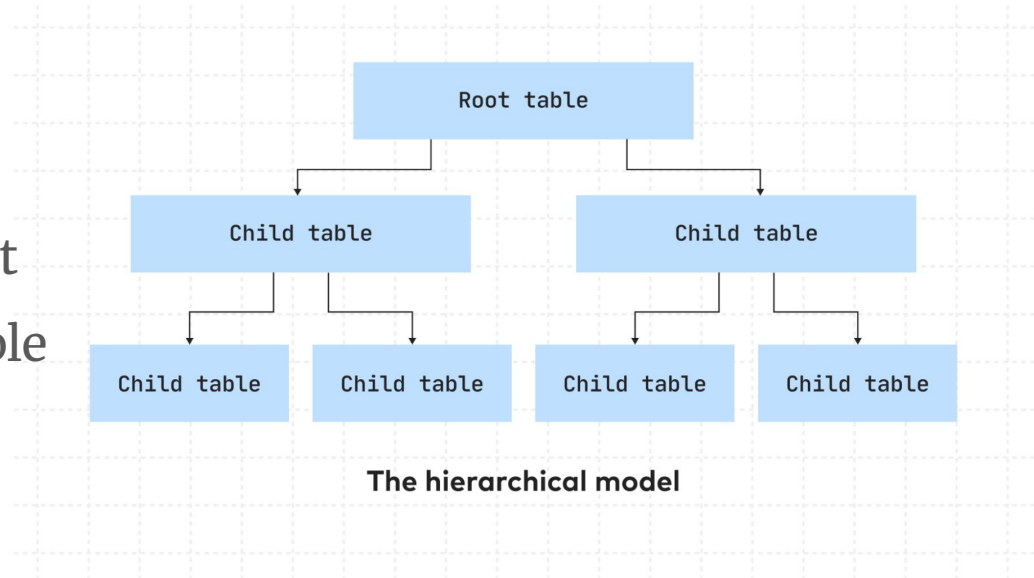
- Relational
- Object Model
- Hierarchical
- Network
- Graph Data Models
- Entity-Relationship

Data Model | Hierarchical Model

A hierarchical data model organizes data in a tree-like structure, where each record (node) has a single parent (except for the root) and can have multiple children.

Adv: simple, fast

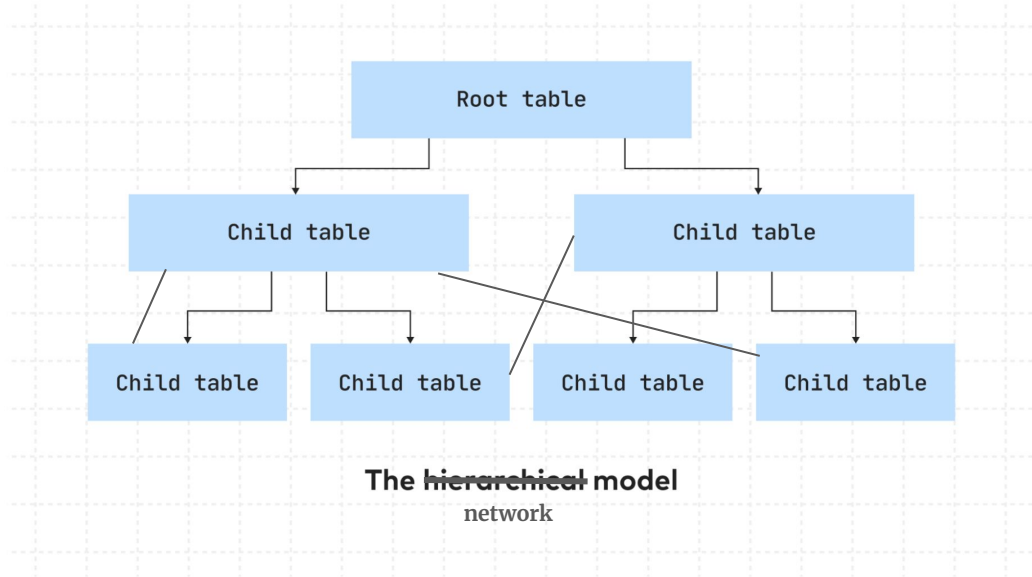
Disadv: inflexible, data redundancy



Data Model | Network Model

A network data model is a flexible way to represent data and relationships, especially useful for complex, many-to-many connections.

It uses a graph structure with nodes (representing entities) and edges (representing relationships) to organize data, allowing for more efficient and direct access paths compared to hierarchical models.



Hierarchical vs Network Model

Feature	Hierarchical	Network
Structure	Tree	Graph
Relationship	1:N only	M:N allowed
Parent Limit	Single	Multiple
Access	Path navigation	Pointer navigation
Flexibility	Low	Medium
Complexity	Low	High

Relational model

Data Model | Relational Model

It is a database model that organizes data into tables (relations) consisting of rows (tuples) and columns (attributes), where relationships between data are represented through values in the tables rather than physical links.

User					
UserID	User	Address	Phone	Email	Alternate
1	Alice	123 Foo St.	12345678	alice@example.org	alice@neo4j.org
2	Bob	456 Bar Ave.		bob@example.org	
...	...	...	...	...	...
99	Zach	99 South St.		zach@example.org	

Order	
OrderID	UserID
1234	1
5678	1
...	...
5588	99

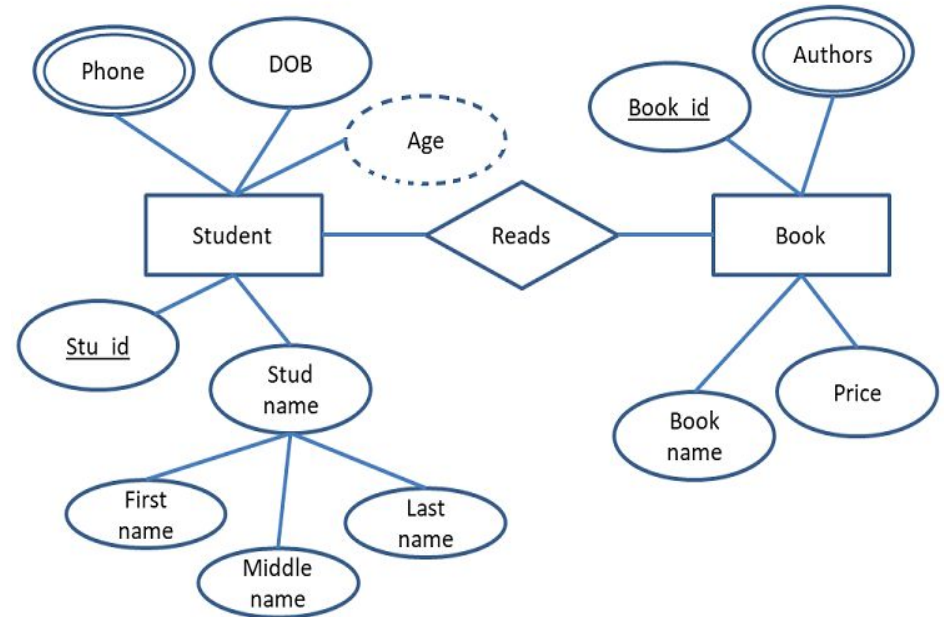
LineItem		
OrderID	ProductID	Quantity
1234	765	2
1234	987	1
...	...	...
5588	765	1

Product		
ProductID	Description	Handling
321	strawberry ice cream	freezer
765	potatoes	
...	...	
987	dried spaghetti	

Data Model | Entity-Relationship Model

The Entity Relationship Model is a conceptual data model that represents the structure of a database using three main components:

- Entities - Real-world objects
- Attributes - Properties of entities
- Relationships - Associations between entities



ENTITY

Any distinguishable 'thing' or 'object' in the real world is called an entity.

It has a set of attributes that may help us to identify it uniquely.

ATTRIBUTES

Attributes are set of properties or characteristics that describe an entity.

E.g. Attributes of a student may include roll_no, name, grade, etc.

Therefore, each entity has a value for an attribute.

TYPES OF ENTITY

In DBMS we can find two types of entity that are distinguished on the basis of their ability to exist independently. They are:

1. Strong entity
2. Weak entity

TYPES OF ENTITY

- **Strong entity**

An entity that does not depend on any other entity for its existence.

It has its own unique identifier (primary key).

It represents the main objects in your database.

E.g. Student can be identified using their roll no.

TYPES OF ENTITY

- **Weak entity**

An entity that depends on a Strong Entity for its existence.

It cannot be uniquely identified by its own attributes alone.

E.g. Emergency contacts are dependent on student entity.

Therefore, it represents contextual or relational data.

TYPES OF ATTRIBUTES

1. Simple

Can't be broken down

2. Composite

Made up of multiple other attributes (e.g name = first_name + last_name)

3. Multivalued

4. Derived

5. Weak key attribute

ENTITY SETS

Set of entities of the same type and sharing the **same** properties (or attributes) is called as an Entity set.

Then, the set of all students at an institution, for example, can be defined under one entity set '*student*'.

A database therefore includes a collection of entity sets, each of which contains any number of entities of the same type

Question

In a Car entity set, would a Toyota Camry and a Ford F-150 be considered the "same type"? Why or why not?

Answer:

Yes, a Toyota Camry and a Ford F-150 would be in the same Car entity set because:

- ✓ Same entity type = "Car"
- ✓ Same attributes = VIN, make, model, year, color, etc.
- ✓ Different values for those attributes

Relationships in E-R diagram

What is a relationship?

-> Association between two entities is called relationship.

It's types are:

1. Strong relationship

Relationship between two strong entity

-> represented by a diamond shape

2. Weak relationship

Relationship between strong and weak entity

-> represented by double diamond shape

Cardinality in E-R diagram

What is a relationship?

-> specifies the numerical relationship between entities

It's types are:

1. One to one relationship
Citizen -> citizenship
2. One to many relationship
University -> students
3. Many to one relationship
Students -> teacher
4. Many to many relationship
Students -> courses

Participation in Relationships

Participation in relationships refers to whether every entity must be involved in a relationship or not.

If every instance of an entity must be related, it's called total participation (mandatory), shown with a double line in ER diagrams.

If some instances may not be related, it's called partial participation (optional), shown with a single line.

Participation in Relationships

- **Total (Mandatory) Participation**

Meaning: Every entity in the set must participate in the relationship.

Symbol in ER diagram: Double line connecting the entity to the relationship diamond.

Example: In Enrolls(Student, Course), if every student must enroll in at least one course, then the Student entity has total participation in Enrolls.

Participation in Relationships

- **Partial (Optional) Participation**

Meaning: Not every entity instance must participate in the relationship.

Symbol in ER diagram: Single line.

Example: An Instructor may not necessarily teach a course in a given semester — so Instructor has partial participation in the Teaches relationship.

Question

Design an ER diagram for a simple library system with these 3 entities: Student, Book, Library

- A student can borrow multiple books
- A book can be borrowed by multiple students
- Each student has a unique student ID
- Each book has a unique ISBN number
- Each loan has a borrow date and due date

Extended ER Terminologies

1. Specialisation

Breaking entity into smaller entities to divide responsibility

2. Generalisation

Taking up roles from multiple smaller entities to the main entity

3. Aggregation

We include multiple entities relating to another entity together and tie the relationship to their collective set

Aspect	Generalization	Specialization	Aggregation
Approach	Bottom-up	Top-down	Relationship abstraction
Purpose	Merge similar entities	Divide entity into sub-entities	Combine entities & relationships
Hierarchy	Merges entities	Creates sub-entities	Represents composite relationships
Example	Student + Teacher → Person	Account → Savings/Current	Employee + Project → Higher entity

Question

Design an ER diagram for a Company Management System with 2 entities:

EMPLOYEE & DEPARTMENT

Requirements:

- Include examples of all 4 types of attributes (simple, composite, derived, multi-valued) in each entity
- Show a meaningful relationship between the entities
- Indicate cardinality between entities

Question #2

Design an ER diagram for a Library Management System with 2 entities:

Library & Books

- Requirements:
- Include examples of all 4 types of attributes (simple, composite, derived, multi-valued) in each entity
- Show a meaningful relationship between the entities
- Indicate cardinality between entities

Question #3

Identify relevant attributes and construct an ER diagram with proper mapping constraints for a university which has many departments and each department has multiple instructors; one among them is the head of the department. An instructor belongs to only one department, each department offers multiple courses, each of which is taught by a single instructor. A student may enroll for many courses offered by different departments. [2075 Bhadra]

Question #4

“Construct an E-R diagram for a car insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents. Each insurance policy covers one or more cars and has one or more premium payments associated with it. Each payment is for a particular period of time, and has an associated due date, and the date when the payment was received.”

Excerpt From

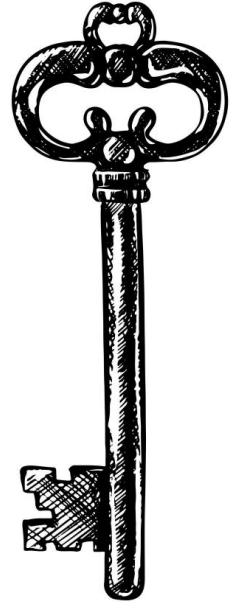
ISE EBook Online for Database System Concepts SILBERSCHATZ;

KEYS

Just like there is a unique key to each lock,

Keys in DBMS help to uniquely identify each entity and thus, distinguish it from other entities of an entity set.

E.g in the entity set 'student', we can use their college roll number as a key.



TYPES OF KEYS | Candidate Key

It is a set of one or more columns in a database table that uniquely identifies every single row in that table, with no unnecessary columns included.

It follows the following rules:

1. Every combination of values in these columns is different for each row
2. No column can be removed from this set without breaking rule #1

Student_ID	Email	Name	Phone
101	alice@school.com	Alice	555-0101
102	bob@school.com	Bob	555-0102
103	carol@school.com	Carol	555-0103

What could be candidate keys?

YES: Student_ID, Email

NO: Name, Phone, (Student_ID, Email)

Simple Candidate Key

```
CREATE TABLE students (  
    student_id INT UNIQUE,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Column Level Defn.

```
CREATE TABLE students (  
    student_id INT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    UNIQUE (student_id)  
);
```

Table Level Defn.

Composite Candidate Key

```
CREATE TABLE student(  
    name varchar(25),  
    roll int,  -- No individual unique constraint  
    class int,  
    unique (class, roll)  -- Roll must be unique within each class  
);
```

TYPES OF KEYS | Primary Key

A primary key is a special type of candidate key that is chosen as the main method of uniquely identifying records in a relational database table.

It follows the following rules:

1. It must be unique
2. It must not be null
3. There must only be one primary key per table

Simple Primary Key

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Column Level Defn.

```
CREATE TABLE students (  
    student_id INT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    PRIMARY KEY (student_id)  
);
```

Table Level Defn.

Composite Primary Key

```
CREATE TABLE order_items (  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    price DECIMAL(10,2),  
    PRIMARY KEY (order_id, product_id) -- Must be table-level  
);
```

TYPES OF KEYS | Super Key

Any set of columns that can uniquely identify a row (may have extra columns) is called a super key.

Simple rule:

If you can find a row with just those columns →
It's a super key.

Checking a SUPER KEY

```
SELECT name, birth_date, COUNT(*)  
FROM students  
GROUP BY name, birth_date;
```

Name	birth_date	count
John	2000-01-01	2
Alice	1999-05-15	3
Bob	2001-03-10	1

```
SELECT name, birth_date, COUNT(*)  
FROM students  
GROUP BY name, birth_date;  
WHERE COUNT(*) >1;
```

Name	birth_date	count
John	2000-01-01	2
Alice	1999-05-15	3

TYPES OF KEYS | Foreign Key

A foreign key is an attribute (or set of attributes) in one table that references *or points to* the primary key of another table to establish a relationship between them.

- Primary Key: Uniquely identifies rows within its own table
- Foreign Key: Points to a primary key in another table to create relationships between tables

TYPES OF KEYS | Foreign Key

```
CREATE TABLE Customers  
(  
    CustomerID INT  
    PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100)  
);
```

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    OrderDate DATE,  
    CustomerID INT,  
    TotalAmount DECIMAL(10,2),  
    FOREIGN KEY (CustomerID)  
    REFERENCES  
    Customers(CustomerID)  
);
```


TYPES OF KEYS | Foreign Key

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    email VARCHAR(100)  
);
```

```
CREATE TABLE  
emergency_contacts (  
    contact_id INT PRIMARY KEY,  
    student_id INT,  
    contact_name VARCHAR(50)  
NOT NULL,  
    FOREIGN KEY (student_id)  
REFERENCES  
students(student_id)  
);
```

TYPES OF KEYS | Summary

- Primary Key: "This row's unique ID in this table"
- Foreign Key: "This points to another table's row"
- Composite Key: "We need multiple columns to be unique together"
- Candidate Key: "This could be the primary key if we choose it"
- Superkey: "More columns than needed to be unique"

Assignment #2

1. What are the similarities and differences between primary key and candidate key?
2. Describe the following using appropriate definition, examples (and diagrams)
 - a. Mapping cardinality:
 - i. One-to-one relationship
 - ii. One-to-many relationship
 - iii. Many-to-one relationship
 - iv. Many-to-many relationship
 - b. Total and partial participation
 - c. Weak entity sets and strong entity sets
 - d. Discriminator attributes
 - e. Identifying relationship

Assignment #2

3. Construct an E-R diagram for a car insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents. Each insurance policy covers one or more cars and has one or more premium payments associated with it. Each payment is for a particular period of time, and has an associated due date, and the date when the payment was received.
4. Define data independence and explain its significance. What is importance of aggregation in ER design? Discuss with an example.